Databases for Data Science

MODULE 2

Sanjeev Wagh, Manisha Bhende, Anuradha Thakare, ‘Fundamentals of Data Science, CRC Press, 1st Edition, 2022.

Overview

- Structured Query Language (SQL):
  - Basic Statistics
  - Data Munging
  - Filtering
  - Joins
  - Aggregation
  - Window Functions
  - Ordered Data
- Preparing No-SQL:
  - Document Databases
  - Wide-column Databases and
  - Graphical Databases

SQL Fundamentals (Foundations)

- Introduction to Relational Databases
- Tables, Rows, Columns, Primary & Foreign Keys
- SQL Data Types and Constraints
- Basic SELECT statements
- WHERE clause and comparison operators
- Logical operators (AND, OR, NOT)
- Sorting with ORDER BY
- Limiting results (LIMIT, OFFSET)

Outcome: Understand how data is stored and retrieved.

Structured Query Language

- It is a standard language used to communicate with relational databases to store, retrieve, modify, and manage data.

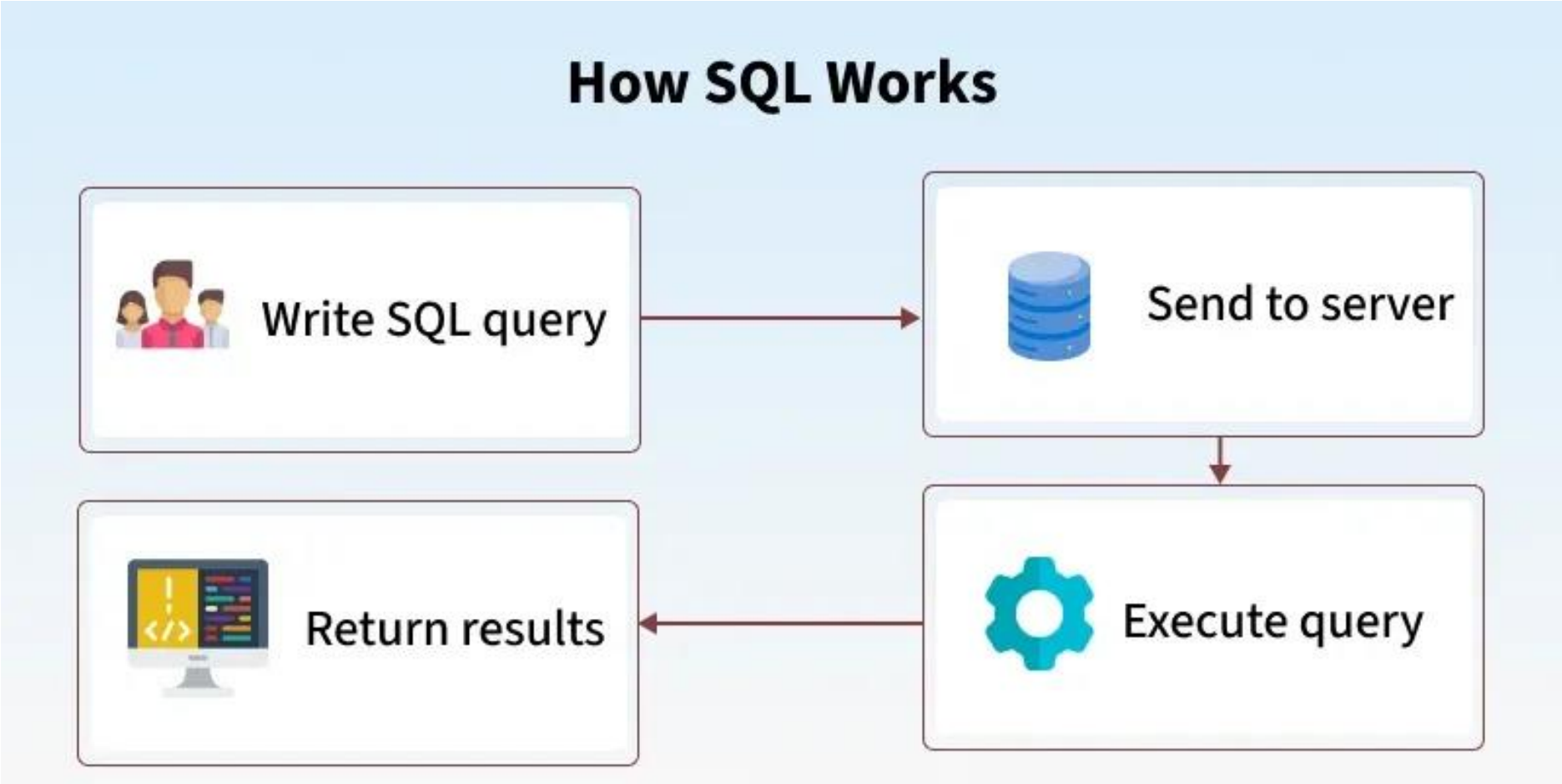
What is SQL?

- SQL is the language you use to ask questions to a database and tell it what to do.

Just like:

- English → communicate with people
- Python/Java → communicate with computers
- SQL → communicate with databases

Structured Query Language



Importance of SQL

- Almost **all applications** use databases (banking, shopping apps, hospitals, universities)
- SQL works with major databases:
  - MySQL
  - PostgreSQL
  - Oracle
  - SQL Server
- Required skill for:
  - Data Science
  - Data Analytics
  - Backend Development
  - AI/ML data handling

What Can SQL Do?

Task	Example
Retrieve data	SELECT
Filter data	WHERE
Insert data	INSERT
Update data	UPDATE
Delete data	DELETE
Combine tables	JOIN
Summarize data	GROUP BY

Types of SQL

- **DDL – Data Definition Language** (Defines database structure)
  - CREATE, ALTER, DROP
- **DML – Data Manipulation Language** (Modifies data)
  - INSERT, UPDATE, DELETE
- **DQL – Data Query Language** (Retrieves data)
  - SELECT
- **DCL – Data Control Language** (Controls access)
  - GRANT, REVOKE
- **TCL – Transaction Control Language** (Manages transactions)
  - COMMIT, ROLLBACK

Introduction to Relational Database Systems (RDBMS)

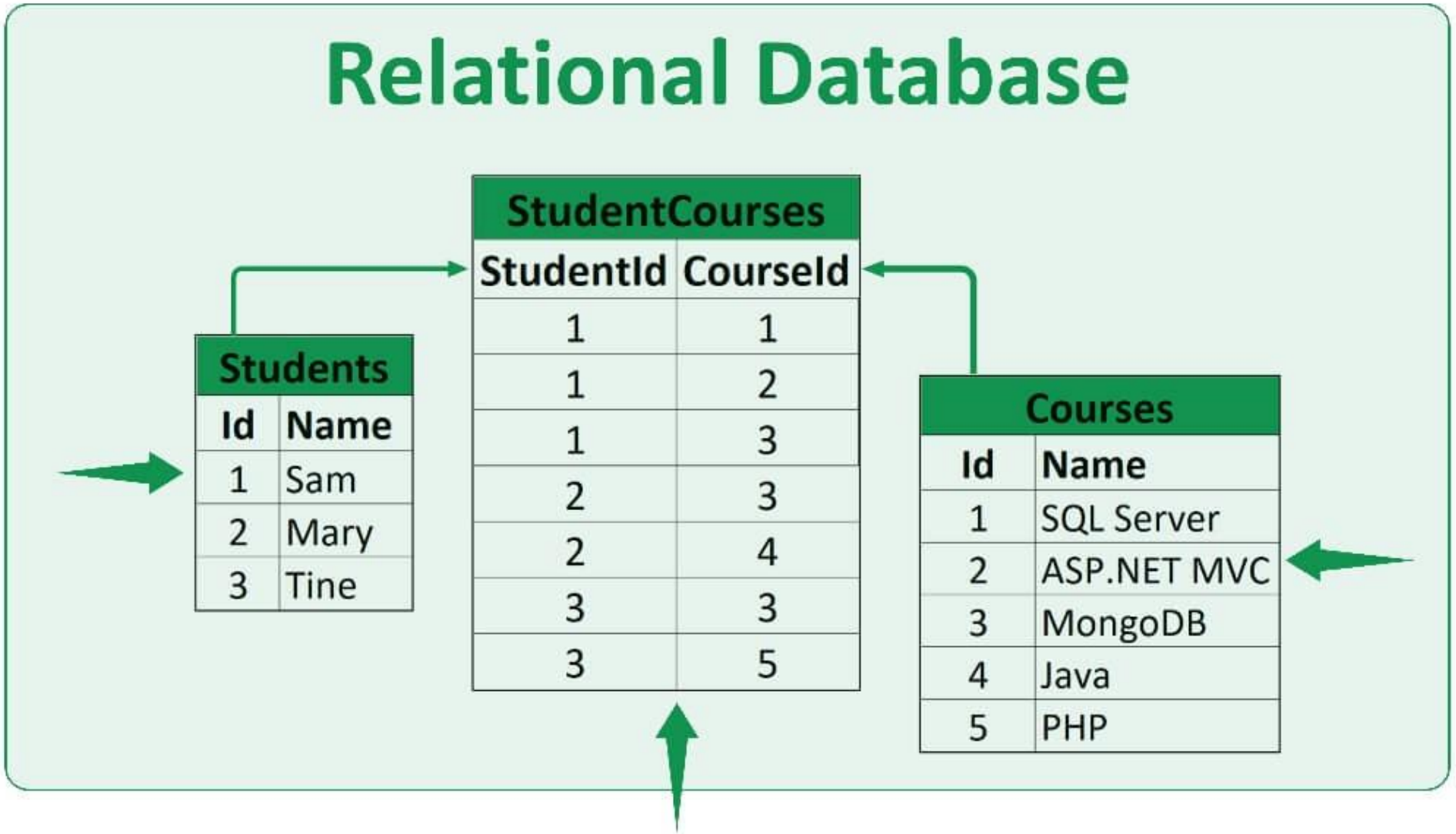
- A **Relational Database Management System (RDBMS)** stores data in **tables** and uses **Structured Query Language (SQL)** to manage and query data.
- **Examples of RDBMS:** MySQL, PostgreSQL, Oracle, SQL Server

Example

A university database may contain tables such as:

- Students
- Courses
- Enrolments

RDBMS


Relational Database
StudentCourses
StudentId CourseId
1 1
1 2
1 3
2 3
2 4
3 3
3 5
Students
Id Name
1 Sam
2 Mary
3 Tine
Courses
Id Name
1 SQL Server
2 ASP.NET MVC
3 MongoDB
4 Java
5 PHP

Relational vs. Non-Relational

- Data Model: Tables with rows and columns, defined schema.
- Schema: Rigid, predefined structure (schema-on-write).
- Query Language: SQL (Structured Query Language).
- Consistency: Strong, ACID compliant (Atomicity, Consistency, Isolation, Durability).
- Scaling: Primarily vertical (scale-up) by adding more power to one server.
- Best For: Structured data, complex joins, financial transactions, data integrity.

Relational vs. Non-Relational

- Data Model: Diverse (document, key-value, graph, wide-column).
- Schema: Flexible, dynamic (schema-on-read).
- Query Language: Database-specific APIs/languages (e.g., MongoDB Query Language).
- Consistency: Often relaxed (BASE), prioritizing availability and partition tolerance over immediate consistency.
- Scaling: Horizontal (scale-out) by adding more servers (sharding).
- Best For: Unstructured/semi-structured data, large volumes, real-time web apps, big data.

Database Structure: Tables, Rows, Columns

- **Table** → Collection of related data
- **Row (Tuple)** → One record
- **Column (Attribute)** → One property of the data

COLUMN (Attributes)

Example Table: Students

Student_ID	Name	Age	Department
101	Anu	20	CSE
102	Ravi	21	ECE

ROW (Tuple)

Primary Keys

Primary Key

- Uniquely identifies each row
- Cannot be NULL or duplicated

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT
);
```

Foreign Keys

- Refers to the primary key of another table

```
CREATE TABLE Enrollments (
        EnrollmentID INT PRIMARY KEY,
        StudentID INT,
        FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

SQL Data Types and Integrity Constraints

Common Data Types

- INT, FLOAT
- VARCHAR(n), CHAR(n)
- DATE

Constraints

- NOT NULL
- UNIQUE
- DEFAULT
- CHECK

```
CREATE TABLE Employees (
        EmpID INT PRIMARY KEY,
        Salary INT CHECK (Salary > 0),
        Email VARCHAR(100) UNIQUE,
        JoiningDate DATE NOT NULL
);
```

Basic SELECT Queries for Data Retrieval

The SELECT statement retrieves data from a table.

```
SELECT * FROM Students;
```

Select specific columns:

```
SELECT Name, Department FROM Students;
```

Data Filtering Using the WHERE Clause

The WHERE clause filters rows based on conditions.

```
SELECT *

FROM Students

WHERE Department = 'CSE';
```

Comparison Operators

Operator	Meaning
=	Equal
!=	Not Equal
<	Less Than
>	Greater Than
<=	Less Than or Equal
>=	Greater Than or Equal

Logical Operators (AND, OR, NOT)

AND

```
SELECT *
FROM Students
WHERE Department = 'CSE' AND Age > 20;
```

OR

```
SELECT *
FROM Students
WHERE Department = 'CSE' OR Department = 'ECE';
```

NOT

```
SELECT *
FROM Students
WHERE NOT Department = 'MECH';
```

DDL – Data Definition Language

- **DDL (Data Definition Language)** is a subset of SQL used to **define, modify, and delete the structure of database objects** such as tables, schemas, and indexes.
- DDL commands affect the database structure, not the data stored inside the tables.
  - CREATE
  - ALTER
  - DELETE

CREATE

- Used to **create database objects** like tables, databases, and views.

```
CREATE TABLE new_table_name AS
    SELECT column1, column2,...
    FROM existing_table_name
    WHERE ....;
```

```
CREATE TABLE Students (
        StudentID INT PRIMARY KEY,
        Name VARCHAR(50),
        Age INT,
        Department VARCHAR(20)
);
```

StudentID	Name	Age	Department

- Creates a new table named Students with specified columns and constraints.

ALTER

- Used to **modify the structure** of an existing table.

Add a New Column

```
ALTER TABLE Students
ADD Email VARCHAR(100);
```

ADD COLUMN

StudentID	Name	Age	Department	Email

Modify a Column

```
ALTER TABLE Students
MODIFY Age INT NOT NULL;
```

MODIFY COLUMN

StudentID	Name	Age	Department	Email

Drop a Column

```
ALTER TABLE Students
DROP COLUMN Email;
```

DROP COLUMN

StudentID	Name	Age	Department

DROP

- Used to **delete database objects permanently**.

Drop a Table

```
DROP TABLE Students;
```

- This removes the table structure and all its data permanently.

DML – Data Manipulation Language

- **DML (Data Manipulation Language)** is a subset of SQL used to **insert, update, and delete data stored in database tables**.
- Unlike DDL, DML affects the actual data (records) inside the tables.
- INSERT
- UPDATE
- DELETE

INSERT

Used to **add new records** into a table.

Insert a Single Record

INSERT INTO Students (StudentID, Name, Age, Department) VALUES (101, 'Anu', 20, 'CSE');

StudentID	Name	Age	Department
101	Anu	20	CSE

INSERT

Insert Multiple Records

INSERT INTO Students

VALUES

(102, 'Ravi', 21, 'ECE’),

(103, 'Meena', 22, 'IT');

StudentID	Name	Age	Department
101	Anu	20	CSE
102	Ravi	21	ECE
103	Meena	22	IT

UPDATE

- Used to **modify existing records** in a table.

Update a Specific Record

UPDATE Students

SET **Age** = 21

WHERE **StudentID** = 101;

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE
103	Meena	22	IT

- Always use WHERE to avoid updating all records.

DELETE

Used to **remove records** from a table.

Delete a Specific Record

DELETE FROM Students

WHERE StudentID = 103;

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE
103	Meena	22	IT

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE

DELETE

Delete All Records (Table remains)

DELETE FROM Students;

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE

Table is deleted

DQL – Data Query Language

- **DQL (Data Query Language)** is a subset of SQL used to **retrieve data from database tables** without modifying the data.
- DQL focuses only on reading and querying data.

- SELECT

SELECT

- Used to fetch data from one or more tables.

```
SELECT column1, column2

FROM table_name;
```

SELECT

Retrieve All Columns

SELECT * FROM Students;

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE
103	Meena	22	IT

SELECT

Retrieve Specific Columns

SELECT Name, Department

FROM Students;

Name	Department
Anu	CSE
Ravi	ECE
Meena	IT

SELECT

Filter Data Using WHERE

```
SELECT *

FROM Students

WHERE Department = 'CSE';
```

StudentID	Name	Age	Department
101	Anu	21	CSE

SELECT

Use Comparison Operators

```
SELECT *

FROM Students

WHERE Age >= 21;
```

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE

SELECT

Use Logical Operators

```
SELECT *

FROM Students

WHERE Department = 'CSE' AND

Age > 20;
```

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE
103	Meena	22	IT

SELECT

Sort Results Using ORDER BY

SELECT *

FROM Students

ORDER BY Age DESC;

StudentID	Name	Age	Department
103	Meena	22	IT
101	Anu	21	CSE
102	Ravi	21	ECE

SELECT

Limit Results

```
SELECT *
FROM Students
ORDER BY Age DESC
LIMIT 2;
```

StudentID	Name	Age	Department
101	Anu	21	CSE
102	Ravi	21	ECE
103	Meena	22	IT

SELECT

Limit Results

SELECT *

FROM Students

ORDER BY Age DESC

LIMIT 2;

Step 1: ORDER BY Age DESC

Sorts students by **Age in descending order**:

StudentID	Name	Age	Department
103	Meena	22	IT
101	Anu	21	CSE
102	Ravi	21	ECE
104	Shiva	25	MECH

SELECT

Limit Results

```
SELECT *
FROM Students
ORDER BY Age DESC
LIMIT 2;
```

Step 2: LIMIT 2

Returns **only the first 2 rows** after sorting.

StudentID	Name	Age	Department
103	Meena	22	IT
101	Anu	21	CSE

DCL – Data Control Language

- **DCL (Data Control Language)** is used to **control access and permissions** on database objects.
- GRANT - Used to **give privileges** to users.
- REVOKE - Used to **remove previously granted privileges**.

GRANT

Syntax

GRANT privilege_name

ON table_name

TO user_name;

Example

GRANT SELECT, INSERT

ON Students

TO user1;

REVOKE

Syntax	**Example**
REVOKE privilege_name	REVOKE INSERT
ON table_name	ON Students
FROM user_name;	FROM user1;

DCL – Data Control Language

Privilege	Meaning
SELECT	Read data
INSERT	Add data
UPDATE	Modify data
DELETE	Remove data
ALL	All privileges

TCL – Transaction Control Language

- **TCL (Transaction Control Language)** is used to **manage transactions** in a database.
  - COMMIT - Saves all changes permanently
  - ROLLBACK - Undoes changes made since the last commit
  - SAVEPOINT - Sets a point to which a transaction can be rolled back

TCL – Transaction Control Language

- COMMIT;
- ROLLBACK;
- SAVEPOINT sp1;
- ROLLBACK TO sp1;

TCL – Transaction Control Language

```
BEGIN;

UPDATE Accounts
SET Balance = Balance - 500
WHERE AccountNo = 101;

SAVEPOINT sp1;

UPDATE Accounts
SET Balance = Balance + 500
WHERE AccountNo = 102;

SAVEPOINT sp2:
COMMIT;
-- Error occurs here----
ROLLBACK TO sp1;

COMMIT;
```

1. Transaction begins
2. ₹500 debited from Account 101
3. SAVEPOINT sp1 is created
4. Credit to Account 102 fails
5. ROLLBACK TO sp1 undoes only the credit
6. Debit remains intact
7. COMMIT saves the transaction

Basic Data Retrieval & Filtering

StudentID	Name	Age	Department	Email
101	Anu	21	CSE	anu@gmail.com
102	Ravi	19	ECE	NULL
103	Meena	22	IT	meena@gmail.com
104	Arjun	20	CSE	NULL
105	Kavya	18	IT	kavya@gmail.com

Pattern Matching

- LIKE - Used to search for a **pattern** in a column.
- IN - Used to match **multiple values**.
- BETWEEN - Used to filter values within a **range (inclusive)**.

LIKE

Symbol	Meaning
%	Any number of characters
_	Exactly one character

SELECT *

FROM Students

WHERE Name LIKE 'A%';

StudentID	Name	Age	Department	Email
101	Anu	21	CSE	anu@gmail.com
104	Arjun	20	CSE	NULL

- Retrieves students whose names start with ‘A’.

LIKE

Symbol	Meaning
%	Any number of characters
_	Exactly one character

SELECT *

FROM Students

WHERE Name LIKE '_a%';

StudentID	Name	Age	Department	Email
102	Ravi	19	ECE	NULL
105	Kavya	18	IT	kavya@gmail.com

- Second letter is ‘a’

IN

Used to match **multiple values**.

```
SELECT *
FROM Students
WHERE Department IN ('CSE', 'IT');
```

- Retrieves students from **CSE or IT**.

StudentID	Name	Age	Department	Email
101	Anu	21	CSE	anu@gmail.com
103	Meena	22	IT	meena@gmail.com
104	Arjun	20	CSE	NULL
105	Kavya	18	IT	kavya@gmail.com

BETWEEN

Used to filter values within a **range (inclusive)**.

```
SELECT *
FROM Students
WHERE Age BETWEEN 20 AND 22;
```

- Retrieves students aged **20 to 22**.

Student ID	Name	Age	Department	Email
101	Anu	21	CSE	anu@gmail.com
103	Meena	22	IT	meena@gmail.com
104	Arjun	20	CSE	NULL

Handling NULL Values

- NULL represents **missing or unknown data**.
- **Check for NULL**

SELECT *

FROM Students

WHERE Email IS NULL;

StudentID	Name	Age	Department	Email
102	Ravi	19	ECE	NULL
104	Arjun	20	CSE	NULL

Handling NULL Values

- NULL represents **missing or unknown data**.
- **Check for NOT NULL**

SELECT *

FROM Students

WHERE Email IS NOT NULL;

Student ID	Name	Age	Department	Email
101	Anu	21	CSE	anu@gmail.com
103	Meena	22	IT	meena@gmail.com
105	Kavya	18	IT	kavya@gmail.com

Conditional Expressions (CASE)

- Used to apply **if–else logic** inside SQL queries.

- **Example: Categorize students by age**

```
SELECT Name, Age,
CASE
   WHEN Age >= 22 THEN 'Senior'
   WHEN Age >= 20 THEN 'Junior'
   ELSE 'Freshers'
END AS Category
FROM Students;
```

Name	Age	Category
Anu	21	Senior
Ravi	19	Junior
Meena	22	Senior
Arjun	20	Junior
Kavya	18	Freshers

Simple Subqueries

A **subquery** is a query written **inside another query**.

Example 1: Students older than average age

```
SELECT *
FROM Students
WHERE Age > (
        SELECT AVG(Age)
        FROM Students
);
```

Average Age = (21 + 19 + 22 + 20 + 18) / 5 = 20

StudentID	Name	Age	Department
101	Anu	21	CSE
103	Meena	22	IT

Simple Subqueries

Example 2: Students from departments with more than 2 students

```
SELECT *
FROM Students
WHERE Department IN (
        SELECT Department
        FROM Students
        GROUP BY Department
        HAVING COUNT(*) > 2
);
```

Department	COUNT(*)
CSE	2
IT	2
ECE	1

Empty set (no rows returned)

If a subquery returns no values, the outer query using IN also returns no rows.

Simple Subqueries

- If You Want a Non-Empty Result

Change the condition to: HAVING COUNT(*) >= 2

StudentID	Name	Age	Department
101	Anu	21	CSE
104	Arjun	20	CSE
103	Meena	22	IT
105	Kavya	18	IT

Basic Statistics Using SQL

SQL provides several **built-in aggregate functions** that can be used to compute basic statistical measures directly on the database.

EmpID	Name	Department	Gender	Experience	Salary
101	Amit	IT	M	2	3000
102	Nisha	HR	F	4	4000
103	Yogesh	IT	M	3	3000
104	Puja	HR	F	6	6000
105	Anil	IT	M	5	5000
106	Shiv	IT	M	7	7000
107	Risha	HR	F	2	4000
108	Ram	IT	M	8	8000
109	Husain	IT	M	4	5000
110	Kumar	HR	M	10	10000

MEAN (AVERAGE)

The **mean** is calculated by dividing the sum of all values by the total number of values in the dataset.

SQL Function Used: AVG()

```
SELECT AVG(ColumnName) AS Mean
FROM TableName;
```

- This query computes the average value of ColumnName.

MEAN (AVERAGE)

```
SELECT AVG(Salary) AS

Mean_Salary

FROM Emp;
```

Salary
1000

MODE

- The **mode** is the value that appears **most frequently** in a dataset.
- SQL does **not** provide a built-in mode function, so it must be computed using **GROUP BY** and **COUNT()**

```
SELECT TOP 1 ColumnName
FROM TableName
GROUP BY ColumnName
ORDER BY COUNT(*) DESC;
```

- This query returns the value with the highest frequency.

MODE

```
SELECT Salary

FROM Emp

GROUP BY Salary

ORDER BY COUNT(*) DESC

LIMIT 1;
```

MODE

Step 1: GROUP BY Salary

- Rows are grouped by unique salary values.

Salary	COUNT(*)
3000	2
4000	2
5000	2
6000	1
7000	1
8000	1
10000	1

MODE

Step 2: ORDER BY COUNT(*) DESC

Groups are sorted by frequency (highest count first).

Top frequencies = 2
Salaries with count = 2:

- 3000
- 4000
- 5000

Salary
3000

MEDIAN

The **median** is the middle value of a sorted dataset.

- If the dataset size is **odd**, the median is the middle value.
- If the dataset size is **even**, the median is the average of the two middle values.

- SQL does not provide a direct median function, so it is computed using **row indexing and subqueries**.

MEDIAN

Step 1: Sort Column

```
SELECT Column_name

FROM Table

ORDER BY Column_name;
```

MEDIAN

```
SET @rowindex := -1;

SELECT AVG(t.column_name) AS Median
FROM (
   SELECT @rowindex := @rowindex + 1 AS rownum,
         column_name
   FROM table_name
   ORDER BY column_name
) t
WHERE t.rownum IN (FLOOR(@rowindex/2), CEIL(@rowindex/2));
```

MEDIAN

Step 1: Sort salaries

```
SELECT Salary

FROM Emp

ORDER BY Salary;
```

MEDIAN

```
SET @rowindex := -1;

SELECT AVG(m.Salary) AS Median_Salary
FROM (
   SELECT @rowindex := @rowindex + 1 AS rownum,
         Salary
   FROM Emp
   ORDER BY Salary
) AS m
WHERE m.rownum IN (FLOOR(@rowindex / 2), CEIL(@rowindex / 2));
```

MEDIAN

Median_Salary
5000

```
SELECT

PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY

column_name) AS Median

FROM table_name;
```

Statistics in SQL

- COUNT, SUM, MIN, MAX, MEAN
- RANGE
- VARIANCE & STANDARD DEVIATION
- QUARTILES (Q1, Q2, Q3)
- Z-SCORE (Standardization)
- CORRELATION (Experience vs Salary)
- COVARIANCE
- FREQUENCY DISTRIBUTION
- RUNNING TOTAL & MOVING AVERAGE

EmpID	Name	Department	Experience	Salary
101	Amit	IT	2	3000
102	Nisha	HR	4	4000
103	Yogesh	IT	3	3000
104	Puja	HR	6	6000
105	Anil	IT	5	5000
106	Shiv	IT	7	7000
107	Risha	HR	2	4000
108	Ram	IT	8	8000
109	Husain	IT	4	5000
110	Kumar	HR	10	10000

COUNT, SUM, MIN, MAX, MEAN

```
SELECT
    COUNT(*) AS result_name,
    SUM(Column_name) AS Total_result,
    MIN(Column_name) AS Min_result,
    MAX(Column_name) AS Max_result,
    AVG(Column_name) AS Mean_result
FROM Table_name;
```

COUNT, SUM, MIN, MAX, MEAN

```
SELECT
    COUNT(*) AS Total_Employees,
    SUM(Salary) AS Total_Salary,
    MIN(Salary) AS Min_Salary,
    MAX(Salary) AS Max_Salary,
    AVG(Salary) AS Mean_Salary
FROM Emp;
```

COUNT, SUM, MIN, MAX, MEAN

Total_Employees	Total_Salary	Min_Salary	Max_Salary	Mean_Salary
10	55000	3000	10000	5500

RANGE

Computes the range between minimum and maximum values

```
SELECT MAX(Column) - MIN(Column) AS Result_Range

FROM Table_Name;
```

RANGE

```
SELECT MAX(Salary) - MIN(Salary)

AS Salary_Range

FROM Emp;
```

Salary_Range
7000

VARIANCE & STANDARD DEVIATION

```
SELECT

 VARIANCE(Column) AS Variance,

 STDDEV(Column) AS

Std_Deviation

FROM Table_name;
```

Variance:

$$\sigma^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \mu)^2$$

Standard Deviation:

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2}$$

VARIANCE & STANDARD DEVIATION

```
SELECT

  VARIANCE(Salary) AS Variance,

  STDDEV(Salary) AS

Std_Deviation

FROM Emp;
```

Variance	Std_Deviation
5,250,000	2291

QUARTILES (Q1, Q2, Q3)

- **Quartiles** divide an **ordered dataset** into **four equal parts**.

Steps:

- **Sort** the data in ascending order.
- **Q2 (Median)** = middle value (or average of two middle values).
- **Q1** = median of the **lower half** of the data.
- **Q3** = median of the **upper half** of the data.

Quartile	Meaning
Q1 (First Quartile)	25% of data lies **below** this value
Q2 (Second Quartile / Median)	50% of data lies **below** this value
Q3 (Third Quartile)	75% of data lies **below** this value

QUARTILES (Q1, Q2, Q3)

```
SELECT DISTINCT
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY Salary) OVER ()
        AS Q1,
        PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY Salary)    OVER ()
        AS Median,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY Salary)    OVER ()
        AS Q3
FROM Emp;
```

QUARTILES (Q1, Q2, Q3)

Salaries:

3000, 4000, 3000, 6000, 5000, 7000, 4000, 8000, 5000, 10000

Sorted salaries (ascending):

3000, 3000, 4000, 4000, 5000, 5000, 6000, 7000, 8000, 10000

Q1	Q2 (Median)	Q3
4000	**5000**	**7000**

Z-SCORE (Standardization)

```
SELECT Column_1, Column_2,

(Column_Name - AVG(Column_Name) OVER ()) /

STDDEV(Column_Name ) OVER () AS Z_Score

FROM Table_name;
```

$$\boxed{z = \frac{x - \mu}{\sigma}}$$

Z-SCORE (Standardization)

```
SELECT Name, Salary,

(Salary - AVG(Salary) OVER ()) /

STDDEV(Salary) OVER () AS Z_Score

FROM Emp;
```

$$z = \frac{x - \mu}{\sigma}$$

Z-SCORE (Standardization)

- Mean salary (μ) = 5500
- Standard deviation (σ) ≈ 2291

- **Z > 0** → Salary above average
- **Z < 0** → Salary below average

Name	Salary	Z_Score
Amit	3000	-1.09
Yogesh	3000	-1.09
Nisha	4000	-0.65
Risha	4000	-0.65
Anil	5000	-0.22
Husain	5000	-0.22
Puja	6000	0.22
Shiv	7000	0.65
Ram	8000	1.09
Kumar	10000	1.96

CORRELATION (Experience vs Salary)

```
SELECT CORR(Column_1,

Column_2) AS Correlation_Result

FROM Table_name;
```

Correlation
0.97

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

CORRELATION (Experience vs Salary)

```
SELECT CORR(Experience, Salary)

AS Correlation

FROM Emp;
```

$$r = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum(x_i - \bar{x})^2 \sum(y_i - \bar{y})^2}}$$

Range: **–1 ≤ r ≤ 1**

COVARIANCE

```
SELECT COVAR_POP(Experience,

Salary) AS Covariance

FROM Emp;
```

Covariance
5200

FREQUENCY DISTRIBUTION

```
SELECT Column, COUNT(*) AS Frequency

FROM Table_name

GROUP BY Column

ORDER BY Column;
```

FREQUENCY DISTRIBUTION

```
SELECT Salary, COUNT(*) AS Frequency

FROM Emp

GROUP BY Salary

ORDER BY Salary;
```

FREQUENCY DISTRIBUTION

Salary	Frequency
3000	2
4000	2
5000	2
6000	1
7000	1
8000	1
10000	1

Running Total & Moving Average

Running Total (Cumulative Sum)

- Adds values **row by row**
- Shows the **accumulated total up to the current row**
- Useful for growth tracking, cumulative sales, salaries, etc.

Moving Average

- Calculates the **average over a fixed window of rows**
- Smooths fluctuations
- Common in trend analysis (finance, time series)

Ex: For 3 rows:

Row	Window Rows Used
1	1 row (only current)
2	2 rows (current + 1 previous)
3 onward	3 rows (fixed size)

RUNNING TOTAL & MOVING AVERAGE

```
SELECT Column_1, Column_2,

SUM(Column) OVER (ORDER BY Column) AS Running_Total,

AVG(Column) OVER (ORDER BY Column ROWS 2 PRECEDING) AS

Moving_Avg

FROM Table_naame;
```

RUNNING TOTAL & MOVING AVERAGE

```
SELECT Name, Salary,

SUM(Salary) OVER (ORDER BY Salary) AS Running_Total,

AVG(Salary) OVER (ORDER BY Salary ROWS 2 PRECEDING) AS

Moving_Avg

FROM Emp;
```

RUNNING TOTAL & MOVING AVERAGE

Name	Salary	Running_Total	Moving_Avg
Amit	3000	3000	3000.00
Yogesh	3000	6000	3000.00
Nisha	4000	10000	3333.33
Risha	4000	14000	3666.67
Anil	5000	19000	4333.33
Husain	5000	24000	4666.67
Puja	6000	30000	5333.33
Shiv	7000	37000	6000.00
Ram	8000	45000	7000.00
Kumar	10000	55000	8333.33

Data Munging with SQL

- **Data munging (or data wrangling)** is the process of **cleaning, transforming, and preparing raw data** so it becomes **usable for analysis**.
- In SQL, this is done using **string, numeric, date, and conditional functions**.

Why Data Munging is Needed?

- Raw data is often inconsistent, messy, or incomplete
- Analytics and ML models require clean, well-structured data
- SQL allows munging directly inside the database

Common Data Munging Operations in SQL

- Case Conversion (Standardization)
- Removing Unwanted Spaces
- String Extraction
- Replace / Correct Values
- Handling NULL Values
- Conditional Transformation (CASE)
- Removing Duplicates
- Combining Columns (Data Reshaping)
- Type Conversion

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Case Conversion (Standardization)

UPPER()

SELECT **UPPER**(Column) AS

Name_Upper

FROM Tablename;

LOWER()

SELECT **LOWER**(Column) AS

Name_Lower

FROM Tablename;

Case Conversion (Standardization)

SELECT **UPPER**(Name)

AS Name_Upper,

LOWER(Department) AS

Dept_Lower

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Name_Upper	Dept_Lower
AMIT	it
NISHA	hr
YOGESH	it
PUJA	hr

Removing Unwanted Spaces

TRIM() – removes leading & trailing spaces

SELECT **TRIM**(Column) AS

Clean_Name

FROM Tablename;

LTRIM() / RTRIM()

SELECT **LTRIM**(Column),

RTRIM(Email) FROM Tablename;

Removing Unwanted Spaces

```
SELECT
    UPPER(TRIM(Name)) AS Clean_Name,
    LOWER(TRIM(Department)) AS Clean_Department,
    Salary,
    TRIM(Email) AS Clean_Email
FROM Emp;
```

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Clean_Name	Clean_Department	Salary	Clean_Email
AMIT	it	3000	amit@gmail.com
NISHA	hr	4000	NULL
YOGESH	it	3000	yogesh@gmail.com
PUJA	hr	6000	puja@gmail.com

String Extraction

LEFT() and RIGHT()

SELECT **LEFT**(Column, 3) AS Short_Column_name
FROM Tablename;

SELECT **RIGHT**(Column, 10) AS Short_Column_name
FROM Tablename;

String Extraction

SELECT **LEFT**(Name, 3) AS Short_Name

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Short_Name
ami
NIS
Yog
Puj

SELECT **RIGHT**(Email, 10) AS Email_Domain

FROM Employees;

Email_Domain
@gmail.com
NULL
@gmail.com
@gmail.com

Replace / Correct Values

REPLACE()

SELECT **REPLACE**(Column, ‘from', ‘to') AS Clean_Column

FROM Tablename;

- Useful for **fixing inconsistent labels**.

Replace / Correct Values

SELECT **REPLACE**(Department,

'it', 'IT') AS Clean_Dept

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Clean_Dept
IT
hr
IT
HR

Handling NULL Values

COALESCE() / IFNULL()

SELECT Column, **COALESCE**(Column, 'Not Provided') AS Clean_Column

FROM Tablename;

SELECT Column, **IFNULL**(Column, 'Not Provided') AS Clean_Column

FROM Tablename;

- Prevents NULLs from breaking analysis.

Handling NULL Values

SELECT Name,

COALESCE(Email, 'Not

Provided') AS Email FROM

Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Name	Email
amit	amit@gmail.com
NISHA	Not Provided
Yogesh	yogesh@gmail.com
Puja	puja@gmail.com

Conditional Transformation (CASE)

```
SELECT Name, Salary,
CASE
   WHEN Salary < 4000 THEN 'Low'
   WHEN Salary BETWEEN 4000 AND 6000 THEN 'Medium'
   ELSE 'High'
END AS Salary_Level
FROM Employees;
```

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Name	Salary	Salary_Level
amit	3000	Low
NISHA	4000	Medium
Yogesh	3000	Low
Puja	6000	Medium

Removing Duplicates

SELECT **DISTINCT** Department

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Ensures unique values

Department
it
hr
IT
HR

Combining Columns (Data Reshaping)

SELECT **CONCAT**(Name, ' - ',

Department) AS Employee_Info

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Employee_Info
amit - it
NISHA - hr
Yogesh - IT
Puja - HR

Type Conversion

SELECT **CAST**(Salary AS

FLOAT) AS Salary_Float

FROM Employees;

EmpID	Name	Department	Salary	Email
101	amit	it	3000	amit@gmail.com
102	NISHA	hr	4000	NULL
103	Yogesh	IT	3000	yogesh@gmail.com
104	Puja	HR	6000	puja@gmail.com

Salary_Float
3000.0
4000.0
3000.0
6000.0

JOINS

- SQL joins combine rows from two or more tables based on a related column
- Indication of how data in two sources can be combined based on data values they have in common.

Join - Types

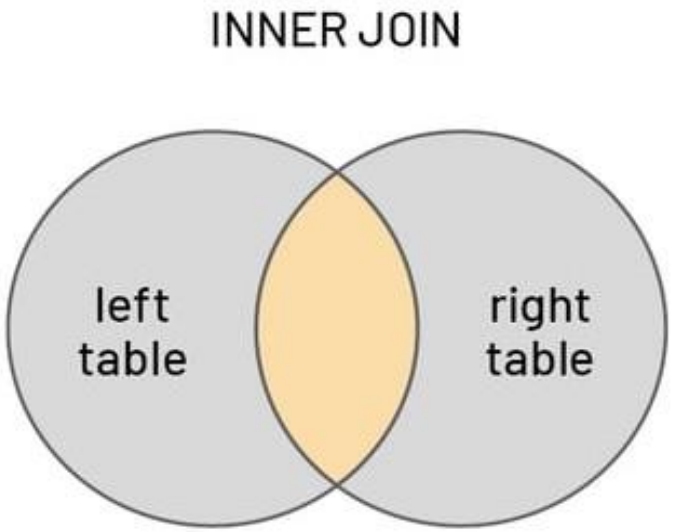
1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL OUTER JOIN
5. CROSS JOIN
6. SELF JOIN

INNER JOIN

- Returns only the rows that have matching values in both tables.

General Syntax

```
SELECT columns
FROM Table1
INNER JOIN Table2
ON Table1.column = Table2.column;
```



Example

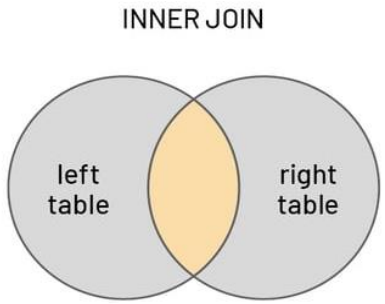


Table: Employees (E)

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT E.Name, D.DeptName
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID;
```

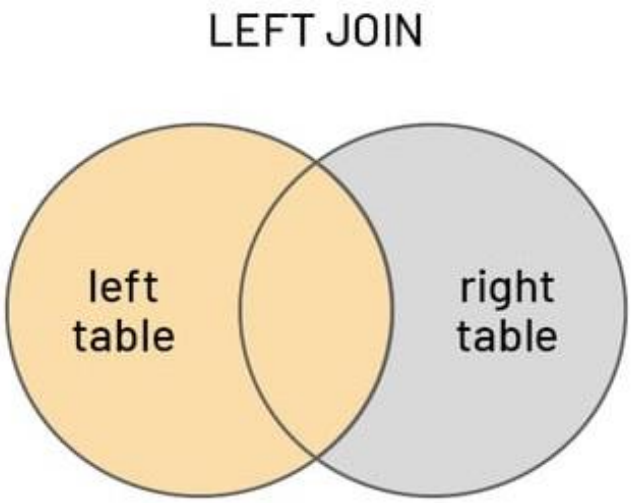
Name	DeptName
Amit	IT
Yogesh	IT
Nisha	HR

LEFT JOIN OR LEFT OUTER JOIN

- Returns **all rows from the left table** and **matching rows from the right table**.

General Syntax

```
SELECT columns
FROM Table1
LEFT JOIN Table2
ON Table1.column = Table2.column;
```



Example

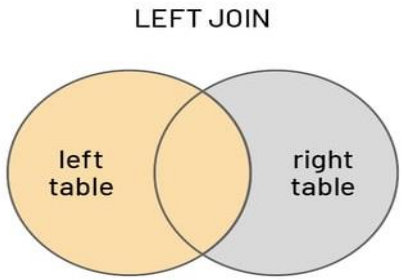


Table: Employees (E)

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department (D)

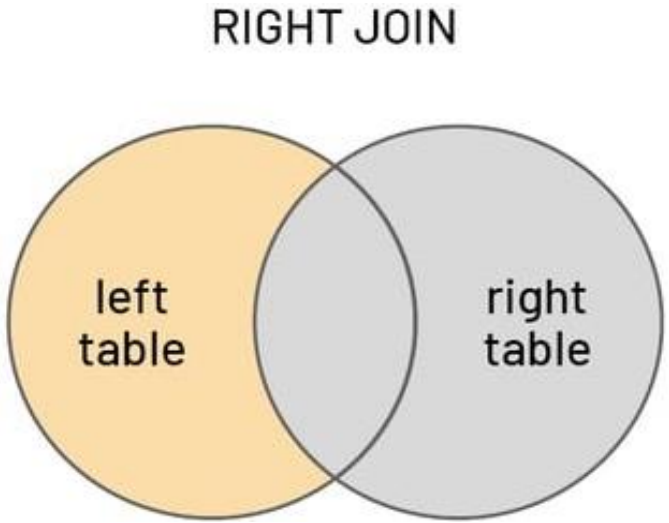
DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT E.EmpID, E.Name, D.DeptName
FROM Employees E
LEFT JOIN Departments D
ON E.DeptID = D.DeptID;
```

EmpID	Name	DeptName
1	Amit	IT
2	Yogesh	IT
3	Nisha	HR
4	Puja	NULL

RIGHT JOIN OR RIGHT OUTER JOIN

- Returns **all rows from the right table** and **matching rows from the left table**.



General Syntax

```
SELECT columns
FROM Table1
RIGHT JOIN Table2
ON Table1.column = Table2.column;
```

Example

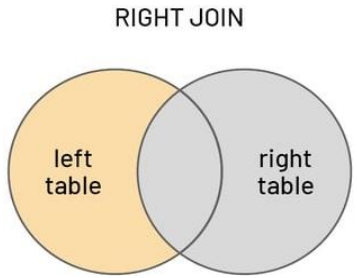


Table: Employees E

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department D

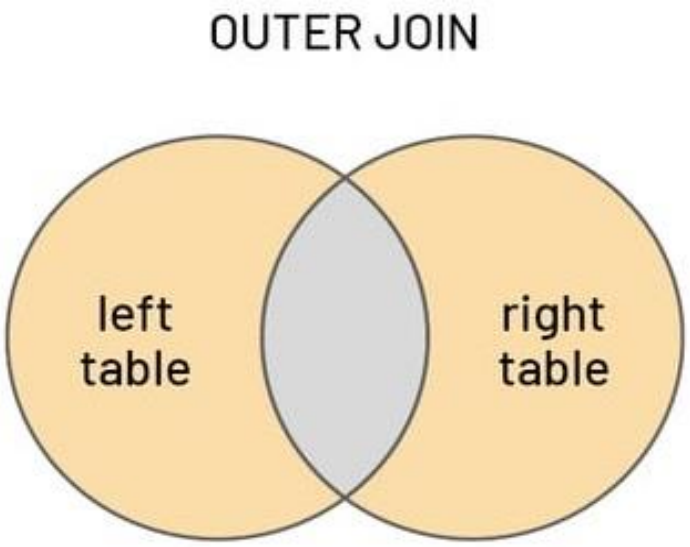
DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT E.Name, D.DeptName
FROM Employees E
RIGHT JOIN Departments D
ON E.DeptID = D.DeptID;
```

Name	DeptName
Amit	IT
Yogesh	IT
Nisha	HR
NULL	Finance

FULL OUTER JOIN

- Returns all rows from both tables. Non-matching rows show NULL.



General Syntax

```
SELECT columns
FROM Table1
FULL OUTER JOIN Table2
ON Table1.column = Table2.column;
```

Example

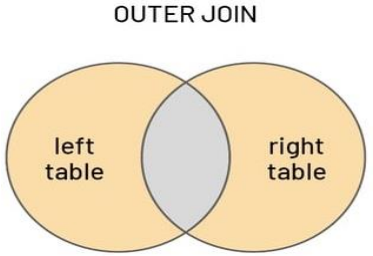


Table: Employees E

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department D

DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT E.Name, D.DeptName
FROM Employees E
FULL OUTER JOIN Departments D
ON E.DeptID = D.DeptID;
```

Name	DeptName
Amit	IT
Yogesh	IT
Nisha	HR
Puja	NULL
NULL	Finance

CROSS JOIN

- Returns Cartesian product (every row of first table with every row of second table).

General Syntax

```
SELECT columns
FROM Table1
CROSS JOIN Table2;
```

Example

Table: Employees E

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department D

DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT E.Name, D.DeptName
FROM Employees E
CROSS  JOIN Departments D;
```

Example

EmpID	Name	DeptName
1	Amit	IT
1	Amit	HR
1	Amit	Finance
2	Nisha	IT
2	Nisha	HR
2	Nisha	Finance
3	Yogesh	IT
3	Yogesh	HR
3	Yogesh	Finance
4	Puja	IT
4	Puja	HR
4	Puja	Finance

SELF JOIN

- A table joined with itself.

General Syntax

```
SELECT A.column, B.column
FROM Table A
JOIN Table B
ON A.column = B.column;
```

Example

Table: Employees

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Table: Department

DeptID	DeptName
10	IT
20	HR
40	Finance

```
SELECT   A.Name AS Employee1,
         B.Name AS Employee2,
         A.DeptID
FROM Employees A
JOIN Employees B
ON A.DeptID = B.DeptID
AND A.EmpID < B.EmpID;
```

Employee1	Employee2	DeptID
Amit	Yogesh	10

Example

- Query: Find employees working in the same dept and show them in pairs.
- Step 1: Employee A (First emp) & Employee B (Second emp) → We are comparing employees with other employees.
- Step 2: A.DeptID = B.DeptID → Only compare employees if they are in the same department.
- Step 3: A.EmpID < B.EmpID → Prevents i. self pairing & ii. Duplicate pairs

Example

- Department 10
- Employees:
  - Amit (1)
  - Yogesh (3)
- Possible pairs:
  - Amit – Yogesh ✅
  - Yogesh – Amit ❌ (blocked because 3 is NOT less than 1)
- Retain:
  - Amit – Yogesh

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Example

Department 20

- Only: Nisha
- No pair possible (only one person).

- Department 30
  - Only: Puja
  - No pair possible.

EmpID	Name	DeptID
1	Amit	10
2	Nisha	20
3	Yogesh	10
4	Puja	30

Example

```
SELECT A.DeptName AS Department1,
    B.DeptName AS Department2
FROM Departments A
JOIN Departments B
ON A.DeptID < B.DeptID;
```

Table: Department

DeptID	DeptName
10	IT
20	HR
40	Finance

Summary

Join Type	Keeps
INNER	Only matches
LEFT	All left + matches
RIGHT	All right + matches
FULL	All from both
CROSS	All combinations
SELF	Compare within same table

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

1. Retrieve the department name along with the number of employees and the average salary for each department, considering only departments that have matching employees using Inner Join.

Example

```
SELECT D.DeptName,
    COUNT(E.EmpID) AS Emp_Count,
    AVG(E.Salary) AS Avg_Salary
FROM Employees E
INNER JOIN Departments D
ON E.DeptID = D.DeptID
GROUP BY D.DeptName;
```

DeptName	Employee_Count	Avg_Salary
IT	3	3666.67
HR	1	4000.00

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

2. Retrieve all employees grouped by department name, including employees whose department does not exist in the Departments table using Left Join.

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

3. Retrieve the department name along with the number of employees and the average salary for each department, considering only departments that have matching employees using Right Join.

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

4. Retrieve all departments and employees, including unmatched records from both tables and group the results by department using Full Join.

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

5. Generate all possible combinations of employees and departments and count how many combinations exist per department using Cross Join.

Example

Table: Employees (E)

EmpID	Name	DeptID	Salary
1	Amit	10	3000
2	Nisha	20	4000
3	Yogesh	10	3000
4	Puja	30	6000
5	Anil	10	5000

Table: Departments (D)

DeptID	DeptName
10	IT
20	HR
40	Finance

6. Find the number of employee pairs working in the same department using Self Join.

Advanced NoSQL for Data Science

- NoSQL databases were introduced in the early 2000s
- Address the limitations of traditional relational databases when handling large-scale, distributed, and unstructured data
- Especially in web, cloud, and social media applications.
- Unlike SQL databases (which store data in fixed tables with predefined schemas), NoSQL databases:
  - Support multiple data models (key-value, document, column-family, graph)
  - Provide schema flexibility
  - Scale horizontally across clusters
  - Follow the BASE model (Basically Available, Soft State, Eventual Consistency) instead of strict ACID guarantees

Why NoSQL?

Limitations of SQL

- Rigid schema (adding new columns is costly)
- Complex joins reduce performance
- Not ideal for large distributed systems
- Difficult to handle unstructured/semi-structured data

Why NoSQL is Preferred in Data Science

- Handles structured, semi-structured, and unstructured data
- Supports real-time big data processing
- Schema-less flexibility
- Designed for distributed clusters
- Efficient for social media, logs, sensor data, streaming data

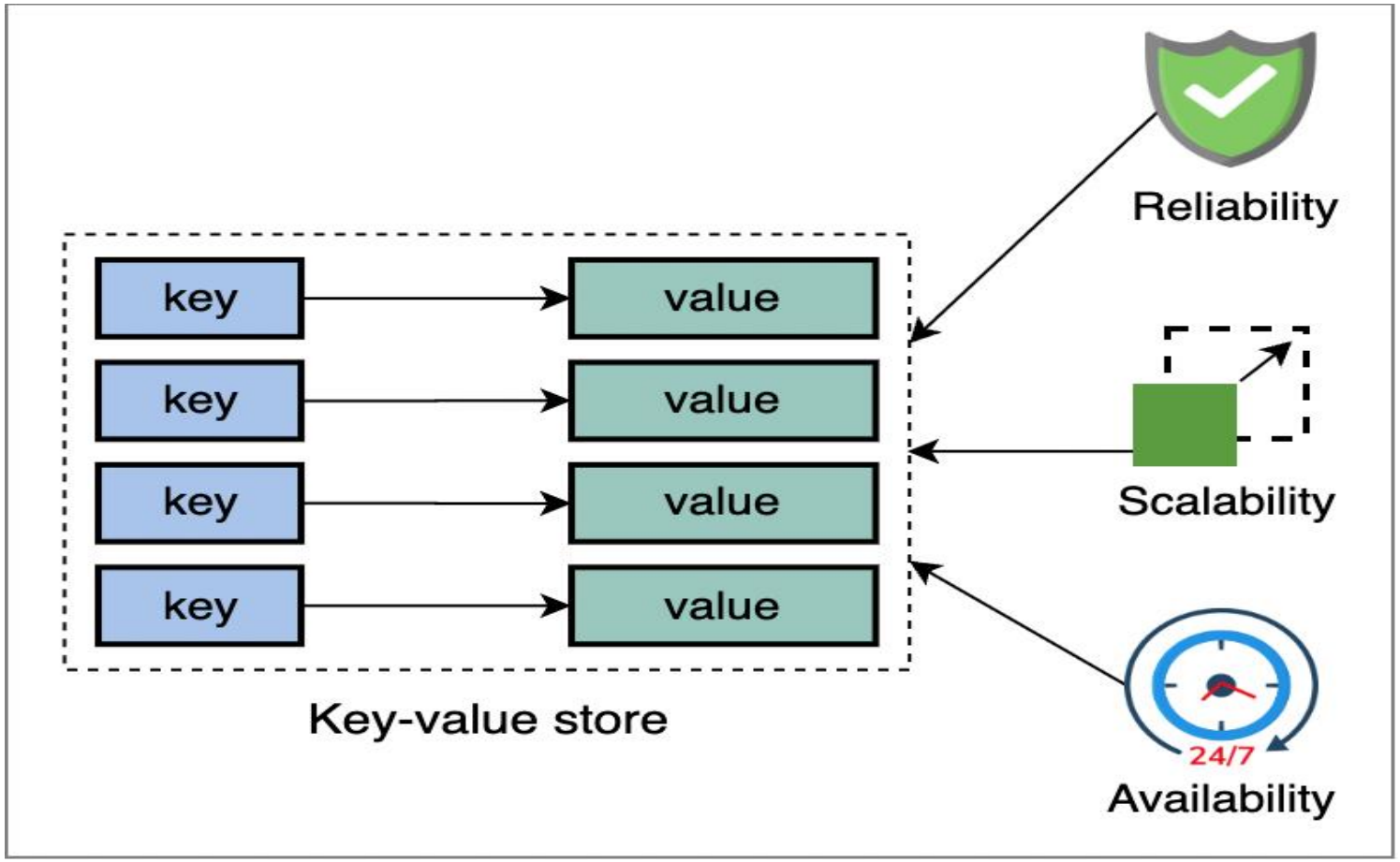
ACID vs BASE Comparison

ACID (SQL)	BASE (NoSQL)
Atomicity	Basically Available
Consistency	Soft State
Isolation	Eventual Consistency
Durability	High Scalability

Types of NoSQL Databases

- NoSQL databases are classified based on **how they store and organize data**.
- The four main types are:
  1. Key-Value Databases
  2. Document Databases
  3. Wide-Column (Column-Family) Databases
  4. Graph Databases

Key-Value Databases



Key-Value Databases

- Store data: as a collection of unique keys and their associated values.

- Structure: Key → Value

- Example:

Values

Key - 101 → {“name”, “Sarah”, “dept”, “SCOPE”, “age”, “20”}

Key - 102 → {“name”, “Jane”, “dept”, “SENSE”, “age”, “19”}

Key - 103 → {“name”, “John”, “dept”, “SCOPE”, “age”, “21”}

Characteristics

- Very fast lookups
- Simple structure
- Highly scalable
- No complex queries

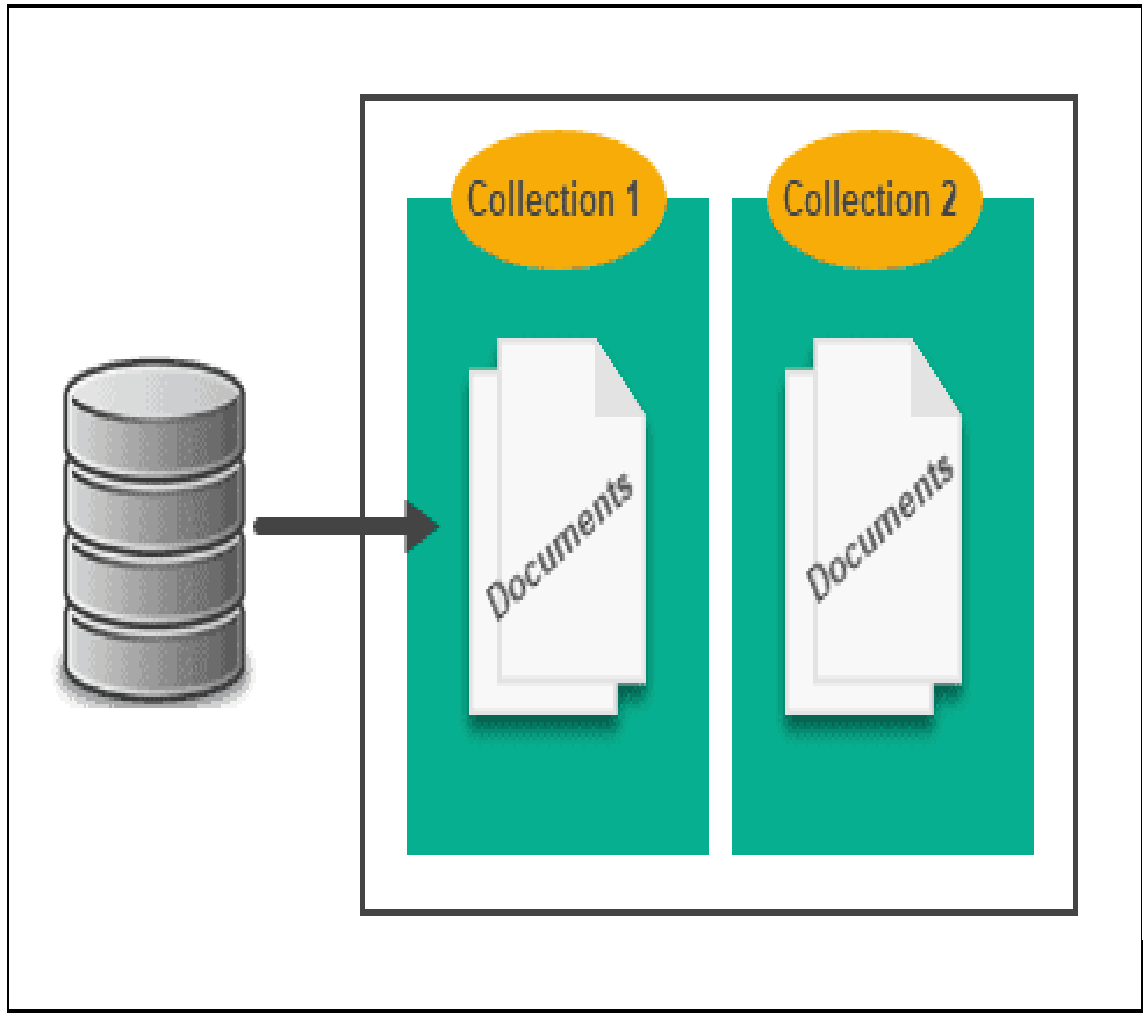
Example

- Redis
- Amazon DynamoDB
- Riak

Use Cases

- Caching
- Session management
- Real-time analytics
- Shopping carts

Document Databases




```
{
    "_id": "tomjohnson",
    "firstName": "Tom",
    "middleName": "William",
    "lastName": "Johnson",
    "email": "tom.johnson@digit
    "department": ["Finance",
    "socialMediaAccounts": [
        {
            "type": "facebo
            "username": "to
        },
        {
            "type": "twitte
            "username": "@t
        }
    ]
}
```

```
{
    "_id": "sammyshark",
    "firstName": "Sammy",
    "lastName": "Shark",
    "email": "sammy.shark@digitalocean.com",
    "department": "Finance"
}
```

```
{
    "_id": "tomjohnson",
    "firstName": "Tom",
    "middleName": "William",
    "lastName": "Johnson",
    "email": "tom.johnson@digitalocean.com",
    "department": ["Finance", "Accounting"]
}
```

Document Databases

- Designed to store, retrieve, and manage information as semi-structured, JSON-like documents
- Structure: Data stored as JSON-like documents.

- Example:

```
{
        "id": 1,
        "name": "John",
        "skills": ["AI", "ML"]
}
```

Each document contains:

- Key-value pairs
- Nested objects
- Arrays

Characteristics

- Flexible schema
- Nested objects supported
- Easy to map with programming languages

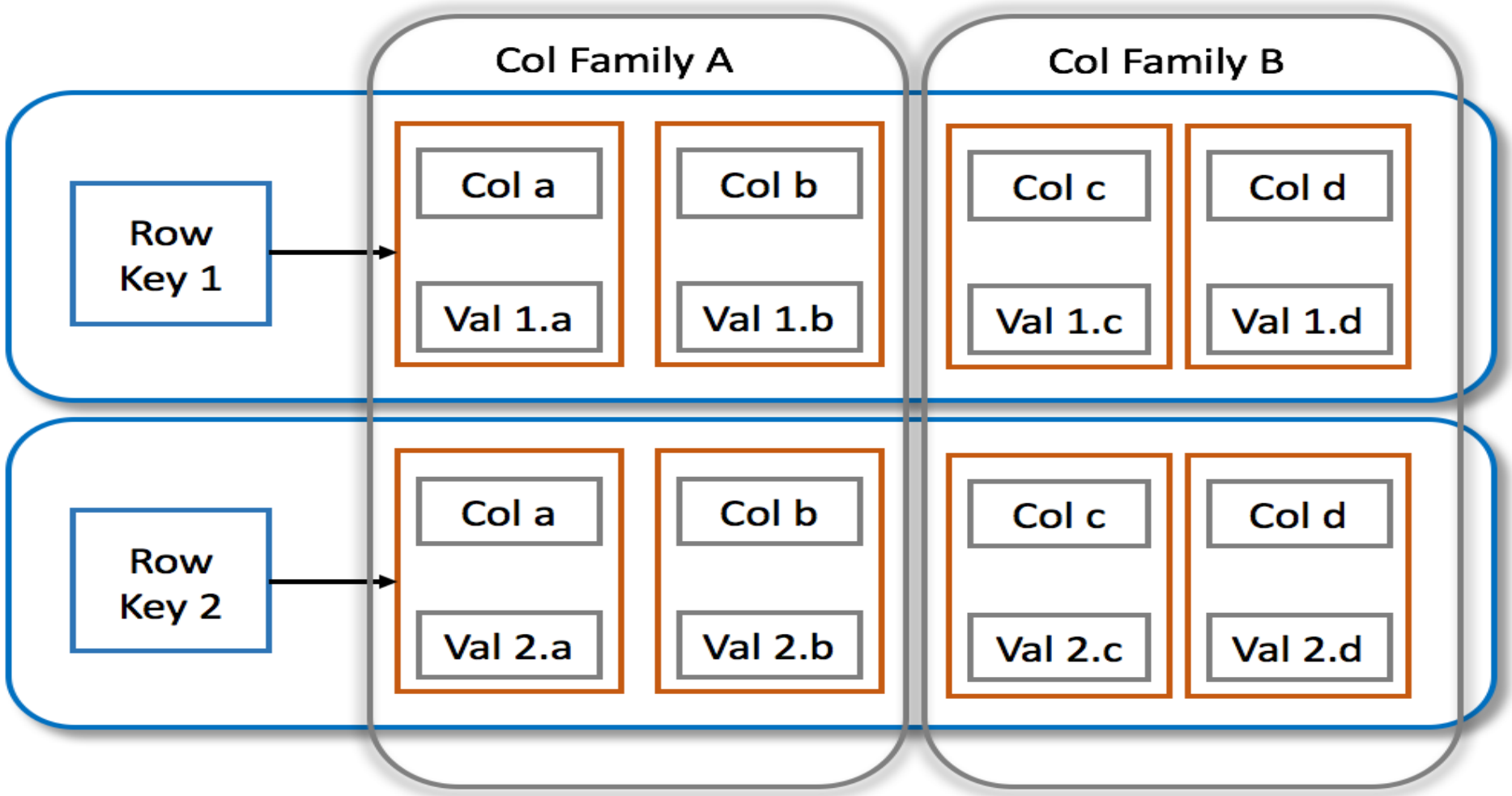
Example

- MongoDB
- CouchDB

Use Cases

- Content management systems
- User profiles
- IoT data
- Big data applications

Wide-Column (Column-Family) Databases



Wide-Column (Column-Family) Databases

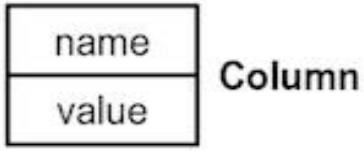
- Organizes data into rows and dynamic columns, grouped into "column families”

- Structure:

  Rows + dynamic columns

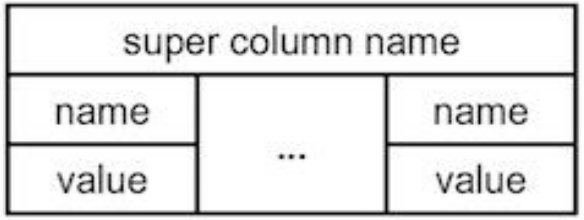
  Columns grouped into column families
- Column
- Super Column
- Column Family
- Super Column Family

Wide-Column (Column-Family) Databases



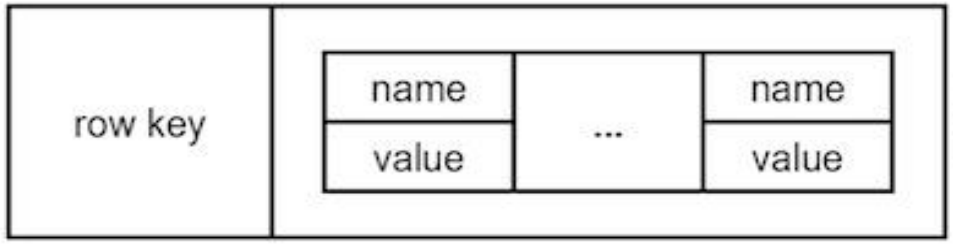
Column

Name → Sarah
Age → 21



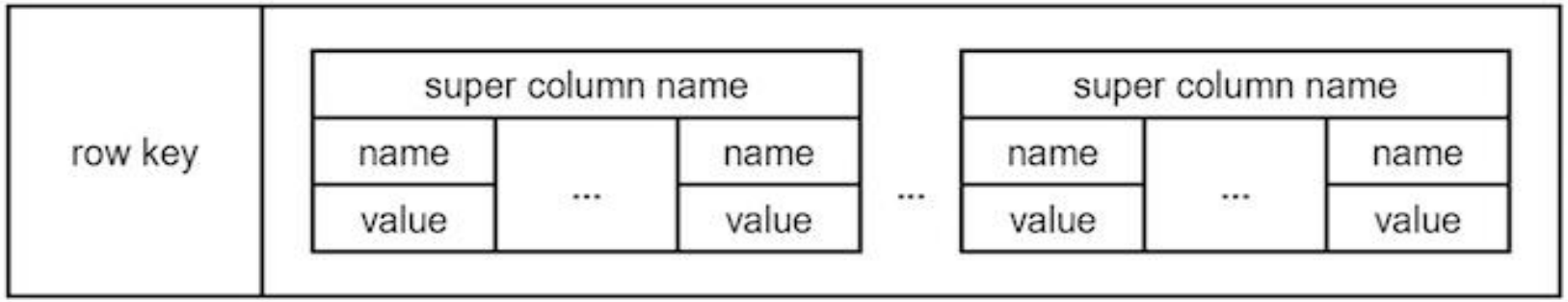
Super Column

Address:
Street → Gandhi Street
City → Adyar
Pincode → 600020



Column Family

Row Key	Name	Dept	Salary
101	Sarah	CSE	50,000
102	Ravi	IT	(no salary column)



Super Column Family

StudentID: 101
Academics:
Subject → AI
Grade → A
Personal:
City → Chennai
Age → 21

Wide-Column (Column-Family) Databases

Term	Meaning
Column	Basic key-value pair
Super Column	Group of columns
Column Family	Collection of columns under a row key
Super Column Family	Collection of super columns under a row key

Characteristics

- Handles massive data
- Distributed architecture
- High write performance

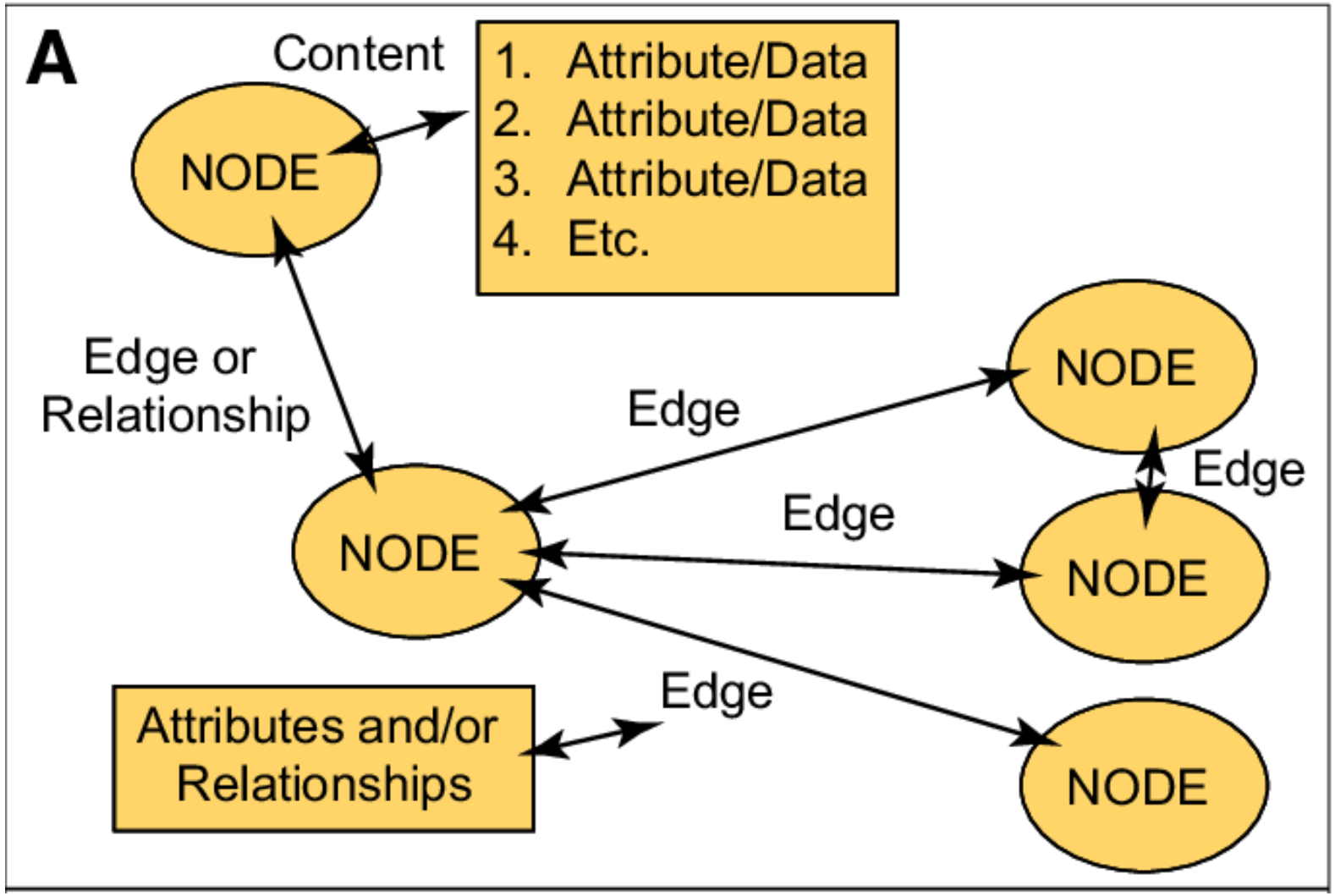
Example

- Apache Cassandra
- Apache HBase

Use Cases

- Time-series data
- Real-time big data analytics
- Financial systems
- Log processing

Graph Databases



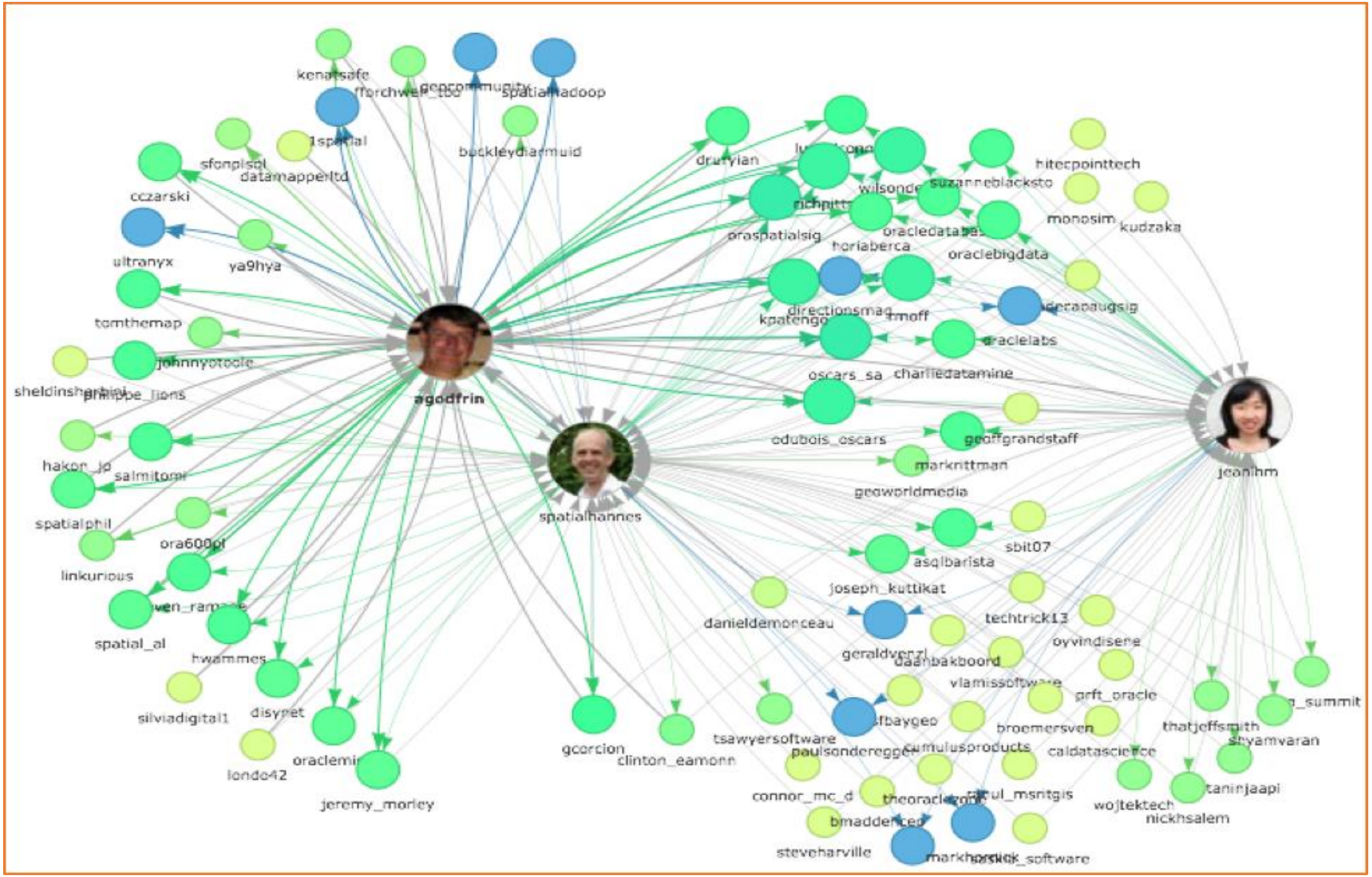
Graph Databases

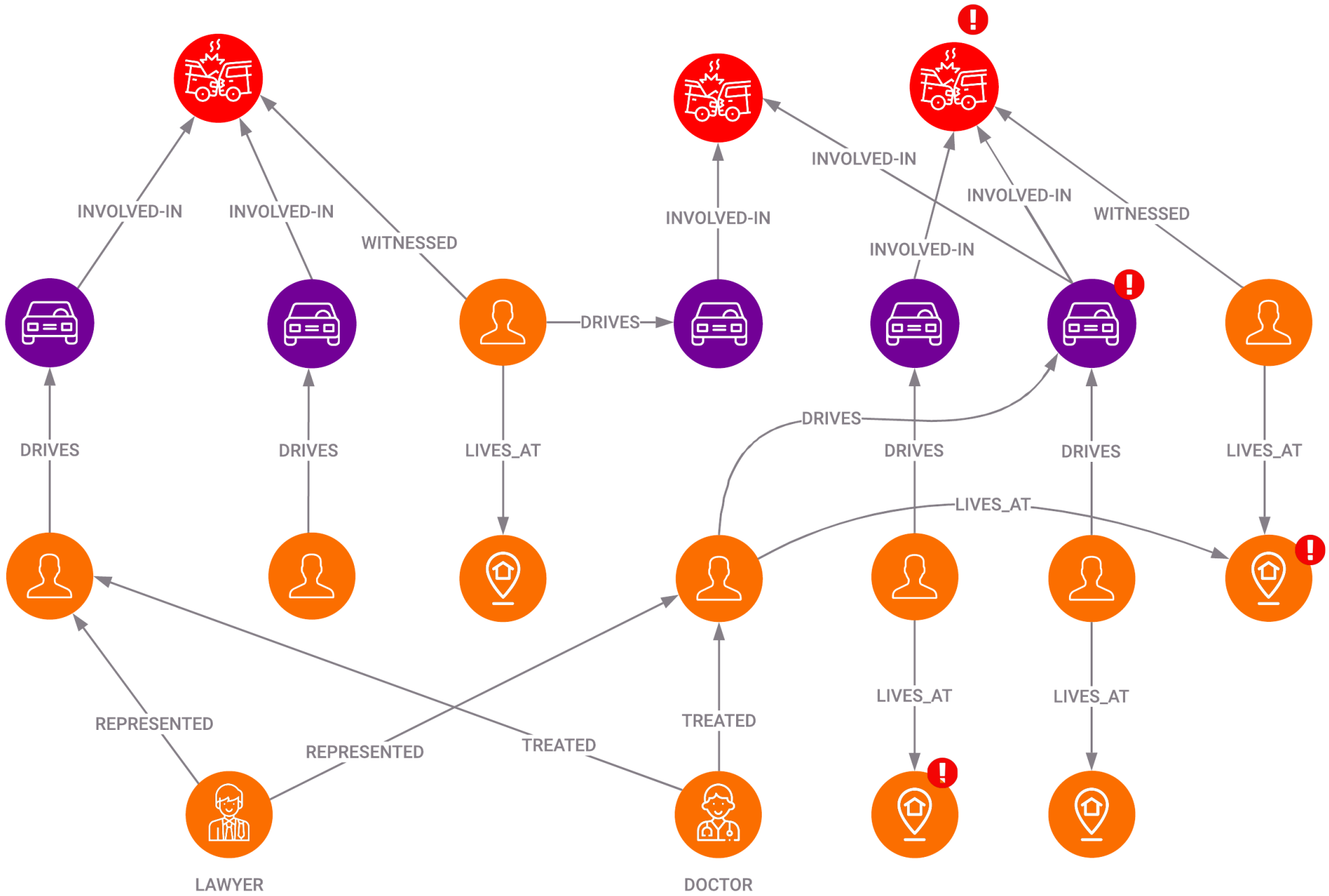
- Graph structures - nodes, edges, and properties to store and represent data
- Prioritize the relationships between data points

- Structure:

  Nodes → Entities

  Edges → Relationships


kenatsafe
spatialhadoop
1spatial
buckleydarmuid
sfonplsql
datamapperltd
cczarski
ultranyx
ya9hya
tomthemap
johnnycoole
philippe_lions
hakon_jo
salmitomi
spatialphil
ora600pl
linkurious
spatial_al
hwammes
silviadigital1
disynet
londo42
oraclemi
jeremy_morley
agodfrin
spatialhannes
gcorcion
clinton_eamonn
druryian
wilsonde
suzannebblacksto
hitecpointtech
monosim
kudzaka
oracledatabas
oraspatialsig
horiaberca
oracledigdata
directionsmag
kpatengo
tmoff
oracelabs
oscars_sa
charliedatamine
cdubois_oscars
geoffgrandstaff
markrittman
geoworldmedia
jeanihm
sbit07
asqlbarista
joseph_kuttikat
danieldemonceau
techtrick13
oyvindisene
geraldverzl
daanbakboord
vlamissoftware
grft_oracle
tsawyersoftware
sfbaygeo
broemersven
thatjeffsmith
shyamvaran
paulsonderegger
cumulusproducts
caldatascience
connor_mc_d
theoracleguy
rasul_msntgis
wojtektech
tanirjaapi
nickhsalem
bmadderlee
steveharville
markhorsick
sasick_software


INVOLVED-IN
INVOLVED-IN
WITNESSED
INVOLVED-IN
INVOLVED-IN
INVOLVED-IN
INVOLVED-IN
WITNESSED
DRIVES
DRIVES
DRIVES
LIVES_AT
DRIVES
DRIVES
DRIVES
LIVES_AT
LIVES_AT
REPRESENTED
REPRESENTED
TREATED
TREATED
LIVES_AT
LIVES_AT
LAWYER
DOCTOR

Characteristics

- Efficient relationship queries
- No complex joins
- Ideal for connected data

Example

- Neo4j
- Amazon Neptune

Use Cases

- Social networks
- Fraud detection
- Recommendation systems
- Knowledge graphs

Summary

Type	Data Model	Best For
Key–Value	Key → Value pairs	Fast lookups
Document	JSON Documents	Flexible schema
Wide-Column	Column families	Big distributed data
Graph	Nodes & Edges	Relationship analysis

Example

- **Q1.** Explain how a Key-Value database stores data. Illustrate with an example suitable for an e-commerce session management system.
- **Q2.** An online shopping website needs to store user session data temporarily. Which type of NoSQL database is most suitable and why?
- **Q3.** Why are Key-Value databases faster than relational databases for simple lookup operations?

Example

- **Q4.** Describe the structure of a document database. Explain how JSON format provides schema flexibility.
- **Q5.** Design a document structure for storing student details including name, subjects, and grades.
- **Q6.** Compare document databases with relational databases in handling semi-structured data.

Example

- **Q7.** What is a column family? Explain with a diagram.
- **Q8.** A bank wants to store millions of transaction records with varying attributes. Which NoSQL model is most suitable? Justify your answer.
- **Q9.** Explain the difference between Column, Super Column, Column Family, and Super Column Family.

Example

- **Q10.** What are nodes and edges in a graph database? Explain with an example.
- **Q11.** A social media platform wants to recommend friends based on mutual connections. Which NoSQL database is best suited and why?
- **Q12.** Why are graph databases more efficient than relational databases for relationship-heavy queries?

VIT.AI

- VIT.AI App Installation Instructions for Student Login

- VIT.AI Student web link - https://vitai.student.edwisely.com

- VIT.AI Student Android link - https://play.google.com/store/apps/details?id=vit.collegestudent.edwisely.com

- VIT.AI Student iOS link - https://apps.apple.com/us/app/vit-ai-student/id6758903987

- Login Process Instructions:-
- 1. Enter your Login ID (Roll Number) to access the platform.
- 2. Click on “Forgot Password” or choose “Sign in with OTP.” An OTP will be sent to your registered email address.
- 3. Enter the OTP received in your email and set a new password for future logins.